“INTELLIGENT COGNITIVE ALARM PLATFORM”

PROJECT REPORT

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP 7.0

(BATCH 3)

Domain: Artificial Intelligence

Submitted by

Lekhana Hugar

Project Mentor

Shanmukha Priya

Mentor – Infosys Internship Program

TABLE OF CONTENTS

Chapter 1	**INTRODUCTION**	**1-3**
1.1	Existing System	1-2
1.2	Proposed System	2
1.2.1	Scope of Work	2-3
1.2.2	Problem Statement	3
Chapter 2	**REQUIREMENT SPECIFICATION**	**4-10**
2.1	Functional Requirements	4
2.2	Non-Functional Requirements	4-5
2.3	Hardware Specification	5
2.4	Software Specification	6
2.5	Technology Stack and Tools Used	6-10
Chapter 3	**METHODOLOGY AND DESIGN**	**11-14**
3.1	Architecture Design	11-12
3.2	Back End	13
3.3	Front End	14
Chapter 4	**SNAPSHOTS**	**15-19**
4.1	Front End	15-17
4.2	Back End	18
4.3	Docker	19
Chapter 5	**DEVELOPMENT (CODE)**	**20**
Chapter 6	**DEPLOYMENT (FRONTEND AND BACKEND)**	**21-22**
6.1	Frontend Deployment	21
6.2	Backend Deployment	21
6.3	Database Deployment	21
6.4	Docker and Docker Compose	22
Chapter 7	**CONCLUSION AND FUTURE ENHANCEMENTS**	**23-24**
7.1	Conclusion	23
7.2	Future Enhancements	23-24
	REFERENCES	**25-26**

Chapter 1

Introduction

The Intelligent Cognitive Alarm Platform is an AI-powered application designed to help users develop consistent wake-up habits and improve their daily routines. Traditional alarm systems mainly notify users at a specified time, but users can easily dismiss or snooze alarms without fully engaging with the wake-up process. This can lead to irregular waking habits and reduced productivity.

The proposed Intelligent Cognitive Alarm Platform introduces cognitive challenges as part of the alarm dismissal process. When an alarm is triggered, the user is required to solve a challenge, such as a mathematical problem, logic puzzle, memory task, word game, pattern recognition problem, riddle, or quiz. The alarm can be dismissed after the submitted answer is verified according to the system's rules.

The platform incorporates artificial intelligence and machine learning techniques to analyze user behavior and challenge performance. It includes adaptive difficulty mechanisms that can adjust challenge complexity based on performance-related information. Behavioral analytics and habit scoring are used to track wake-up consistency, challenge completion, and other relevant user activities. A recommendation system provides guidance to help users improve their waking habits.

The application is developed using React.js for the frontend and FastAPI with Python for the backend. PostgreSQL and MongoDB support data storage, while machine learning libraries support the platform's intelligent features. Firebase Cloud Messaging is integrated for push notifications, and Docker is used to run the application in a containerized environment.

The main aim of this project is to combine alarm management, cognitive engagement, behavioral analysis, and personalized recommendations in a single platform. By making the wake-up process more interactive and data-driven, the system seeks to encourage consistent waking habits and better daily productivity.

1.1 Existing System

Traditional alarm applications provide basic features such as setting alarm times, selecting alarm sounds, and configuring repeat schedules. However, users can easily dismiss or snooze alarms without completing any additional task.

Most conventional alarm systems focus on providing notifications rather than verifying wake-up engagement. They also offer limited tracking of wake-up consistency, snooze patterns and user behavior.

Limitations of the Existing System:

1. Easy dismissal and snoozing of alarms.
2. Limited tracking of waking habits.
3. No integrated cognitive challenge verification in basic alarm workflows.
4. Limited personalization based on user performance.
5. Lack of integrated habit scoring and personalized recommendations.

1.2 Proposed System

The proposed Intelligent Cognitive Alarm Platform combines alarm management with cognitive challenges to encourage consistent waking habits. When an alarm is triggered, the system presents a challenge that the user must solve as part of the wake-up verification process.

The platform supports seven types of challenges: mathematics, logic, memory, word games, patterns, riddles and quizzes. It records challenge results and uses behavioral analytics, adaptive difficulty and habit scoring to track user performance. A recommendation system provides personalized guidance to help users improve their waking routines.

The application uses React.js for the frontend, FastAPI and Python for the backend, PostgreSQL and MongoDB for data storage, and machine-learning components for adaptive intelligence. Firebase Cloud Messaging supports push notifications, while Docker enables local containerized deployment.

1.2.1 Scope of Work

The project covers the following functionalities:

1. User registration, login and profile management.
2. Alarm creation and recurring alarm scheduling.
3. Cognitive challenge generation and answer verification.
4. Adaptive challenge difficulty based on user performance.

5. Behavioral analytics and habit scoring.
6. Personalized recommendations and dashboard visualizations.
7. Firebase Cloud Messaging integration.
8. Application testing and local deployment using Docker.

1.2.2 Problem Statement

Traditional alarm applications mainly provide time-based notifications and allow users to dismiss or snooze alarms easily. They offer limited cognitive engagement and do not sufficiently integrate wake-up verification, behavioral analytics and personalized habit improvement.

Therefore, this project aims to develop an intelligent alarm platform that combines cognitive challenges, adaptive difficulty, habit scoring and personalized recommendations to encourage consistent waking habits and provide meaningful feedback on user performance.

Chapter 2

Requirement Specification

2.1 Functional Requirements

Functional requirements describe the operations and features provided by the Intelligent Cognitive Alarm Platform.

The main functional requirements are:

1. User Authentication: The system allows users to register, log in and access features according to their roles.
2. Profile Management: Users can manage their profiles and wake-up preferences.
3. Alarm Management: Users can create, edit, delete and schedule recurring alarms.
4. Cognitive Challenges: The system provides mathematical, logic, memory, word, pattern, riddle and quiz challenges.
5. Wake-Up Verification: The system evaluates answers and supports retries for incorrect responses.
6. Adaptive Difficulty: The system uses performance information to support challenge difficulty selection.
7. Behavioral Analytics: The system records and analyzes challenge results and wake-up activity.
8. Habit Scoring: The system calculates a score based on selected behavioral factors.
9. Recommendations: The system provides guidance based on recorded user behavior.
10. Dashboard and Notifications: Users can view performance summaries, while Firebase Cloud Messaging supports push notifications.

2.2 Non-Functional Requirements

Non-functional requirements define the quality, reliability, security, and performance expected from the Intelligent Cognitive Alarm Platform.

1. Performance: The system should respond quickly to user actions, alarm operations, and cognitive challenge submissions.

2. Security: User accounts and personal information should be protected through authentication and secure handling of data.

3. Usability: The interface should be simple, clear, and easy to use for managing alarms and viewing performance.

4. Reliability: The system should handle alarm scheduling, challenge verification, and data storage consistently.

5. Scalability: The architecture should support future improvements and accommodate an increasing number of users and records.

6. Maintainability: The application should use a modular structure so that individual components can be updated and debugged easily.

7. Data Integrity: Alarm details, challenge results, user activity, and habit scores should be stored and processed accurately.

8. Compatibility: The web application should work with modern web browsers and support the required frontend and backend environments.

9. Availability: The system should be accessible whenever its required services are running and the user has a working connection.

10. Extensibility: The platform should allow future integration of additional cognitive challenges, machine-learning models, and notification features.

2.3 Hardware Specification

The Intelligent Cognitive Alarm Platform is a web-based application developed using a frontend, backend, and database architecture. The following hardware specifications are recommended for development and execution.

Component	Minimum Specification
Processor	Intel Core i3 or equivalent
RAM	8 GB
Storage	10 GB of available space
System	Laptop or desktop computer
Input Devices	Keyboard and mouse

Display	Standard monitor or laptop display
Internet	Internet connection for online features and notifications

2.4 Software Specification

The Intelligent Cognitive Alarm Platform uses the following software technologies for development, database management, machine learning, and deployment.

Software / Technology	**Purpose**
Operating System	Windows
Frontend	React.js, JavaScript, HTML, CSS
Backend	Python, FastAPI
Database	PostgreSQL and MongoDB
Machine Learning	Scikit-learn, XGBoost
Deep Learning	TensorFlow and PyTorch
Data Processing	Pandas and NumPy
Authentication	JWT (JSON Web Token)
Notifications	Firebase Cloud Messaging (FCM)
Data Visualization	Chart.js and Plotly
Development Tools	Visual Studio Code, Postman
Version Control	Git and GitHub
Deployment	Docker

2.5 Technology Stack and Tools Used

This section describes the technologies used in the Intelligent Cognitive Alarm Platform, their purpose, and their role in the development of the application.

2.5.1 Frontend Technologies

React.js

React.js is a JavaScript library used to develop interactive user interfaces. It is used to build the frontend of the platform, including the dashboard, alarm management pages, cognitive challenge screens, and performance displays. It allows the interface to update dynamically based on user interactions and responses from the backend.

JavaScript

JavaScript is a programming language used to implement frontend functionality. It handles user interactions, form inputs, API requests, and the dynamic display of information.

HTML and CSS

HTML provides the structure of the web pages, while CSS controls their appearance and layout. They help create readable forms, buttons, dashboards, and other interface components.

2.5.2 Backend Technologies

Python

Python is the primary programming language used for backend development and AI-related processing. It supports application logic, data processing, behavioral analysis, and integration with machine-learning libraries.

FastAPI

FastAPI is a Python web framework used to develop backend APIs. It handles requests from the React.js frontend and supports operations such as authentication, alarm management, challenge processing, and retrieving user information.

The frontend communicates with FastAPI endpoints to submit user actions and receive processed results.

2.5.3 Database Technologies

PostgreSQL

PostgreSQL is a relational database management system used to store structured application data. It supports information that can be organized into related tables, such as user accounts,

alarms, and associated records. The backend communicates with PostgreSQL to retrieve and update this information.

MongoDB

MongoDB is a NoSQL database that stores data in document-based structures. It supports flexible data storage for application records where a document-oriented format is suitable.

PostgreSQL and MongoDB serve different data-storage needs within the platform. The exact tables and collections used should correspond to the implemented database configuration.

2.5.4 Artificial Intelligence and Machine Learning

Scikit-learn

Scikit-learn is a Python machine-learning library that provides algorithms for data analysis and prediction. In the platform, it supports machine-learning components related to user performance and adaptive challenge difficulty.

XGBoost

XGBoost is a machine-learning library based on gradient-boosted decision trees. It can be used to analyze performance-related data and support predictive tasks in the adaptive intelligence module.

TensorFlow

TensorFlow is a machine-learning framework used to develop and execute neural-network models. It supports deep-learning experimentation and model-based analysis within the platform.

PyTorch

PyTorch is a deep-learning framework used to implement and execute neural-network models. It supports AI-related development and experimentation with behavioral and performance data.

These technologies provide tools for developing intelligent features. Their inclusion does not itself establish a particular prediction accuracy; model performance should be evaluated using actual test results.

2.5.5 Data Processing Technologies

Pandas

Pandas is a Python library used to organize, manipulate, and analyze structured data. It supports the processing of user activity records and performance information used in analytics.

NumPy

NumPy is a Python library for numerical computing. It provides efficient operations on arrays and numerical values and can support calculations required for analytics and machine-learning processing.

2.5.6 Authentication and Notification Technologies

JSON Web Token (JWT)

JWT is a token-based mechanism used to support authentication and authorization. In the platform, it is used to manage authenticated access to protected backend endpoints.

Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging is a service for sending push notifications to supported client applications and browsers. The platform integrates FCM to support notification functionality. Actual delivery depends on valid configuration, registered tokens, permissions, and the receiving environment.

2.5.7 Data Visualization Technologies

Chart.js

Chart.js is a JavaScript charting library used to present data visually. It can display performance summaries, activity trends, and habit-related information in chart form.

Plotly

Plotly is a data-visualization library that supports interactive charts and graphs. It can be used to present analytical information and trends in a form that is easier to interpret.

Only the visualization libraries actually used by the frontend should be described as implemented features in the final report.

2.5.8 Development and Deployment Tools

Visual Studio Code

Visual Studio Code is a source-code editor used to develop, edit, and debug the application's frontend and backend code.

Postman

Postman is an API testing tool used to send requests to backend endpoints and inspect their responses. It helps verify API behavior during development and debugging.

Git and GitHub

Git is a version-control system used to track changes in source code. GitHub provides an online repository platform for storing and managing project code and supporting collaboration.

Docker

Docker is a containerization platform used to package application components and their dependencies into containers. It helps maintain a consistent development and execution environment.

Docker Compose

Docker Compose is used to configure and run multiple related containers together. In this project, it supports running the frontend, backend, and database services as part of the local deployment setup.

Chapter 3

Methodology and Design

The methodology of the Intelligent Cognitive Alarm Platform involves designing and developing a web-based alarm system that combines cognitive challenges, machine learning, behavioral analysis, and personalized recommendations.

The development process is divided into the following stages:

1. Requirement Analysis: Identify the limitations of traditional alarm systems and define the required features.
2. System Design: Design the frontend, backend, database, and communication between the modules.
3. Frontend Development: Develop the user interface for alarm management, cognitive challenges, dashboards, and activity reports using React.js.
4. Backend Development: Implement APIs, authentication, alarm management, challenge verification, and data processing using Python and FastAPI.
5. Database Integration: Use PostgreSQL and MongoDB to manage the application's data.
6. Adaptive Intelligence: Use machine-learning techniques and behavioral data to support adaptive challenge difficulty and personalized recommendations.
7. Testing and Deployment: Test the application's functionality and run the services using Docker Compose.

This methodology provides a structured approach to developing, integrating, and testing the different components of the platform.

3.1 Architecture Design

The Intelligent Cognitive Alarm Platform follows a client-server architecture consisting of a frontend, backend, database, machine-learning modules, and notification services. These components work together to manage alarms, present cognitive challenges, analyze user behavior, and generate personalized recommendations.

Main Components

1. Frontend Layer: Developed using React.js, this layer provides the user interface for login, alarm management, cognitive challenges, dashboards, and activity reports.

2. Backend Layer: Developed using Python and FastAPI, this layer handles API requests, user authentication, alarm operations, challenge verification, and application logic.

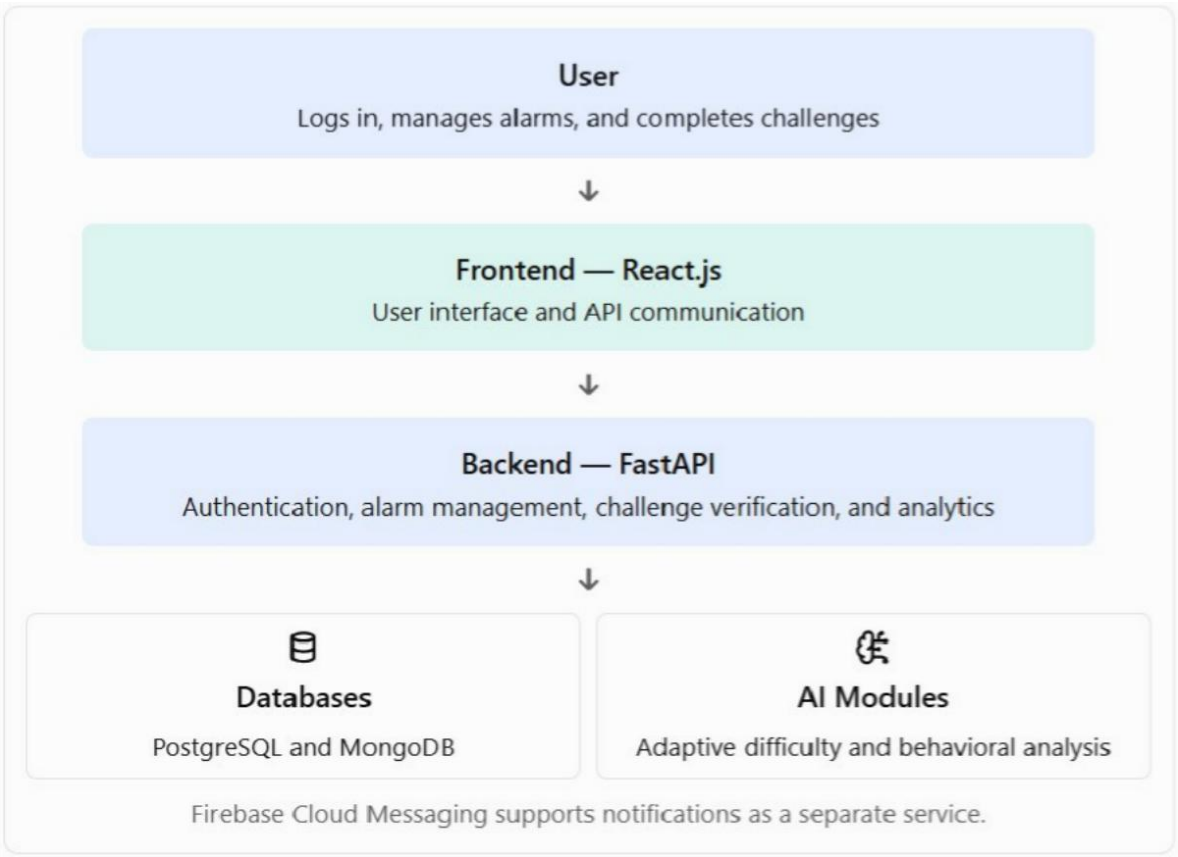
3. Database Layer: PostgreSQL stores structured data such as user information, alarms, and challenge results. MongoDB supports additional data storage and analysis requirements.

4. Machine-Learning Layer: Machine-learning and deep-learning libraries support adaptive challenge difficulty, behavioral analysis, and personalized recommendations.

5. Notification Layer: Firebase Cloud Messaging (FCM) supports sending notifications to registered client devices or browsers, subject to correct configuration and permissions.

6. Deployment Layer: Docker and Docker Compose are used to run the frontend, backend, and database services in containers.

System Workflow



3.2 Back End

The backend of the Intelligent Cognitive Alarm Platform is developed using Python and FastAPI. It acts as the main processing layer between the frontend and the databases. It handles user requests, manages alarms, verifies cognitive challenge answers, processes user performance data, and supports adaptive difficulty and personalized recommendations.

Main Functions of the Backend

1. User Authentication: Handles user registration, login, and authentication using JWT.
2. Alarm Management: Processes alarm creation, updating, deletion, scheduling, and recurring alarm details.
3. Cognitive Challenge Processing: Manages challenge-related operations and verifies user answers.
4. Performance Tracking: Records challenge results, completion time, attempts, and other relevant performance information.
5. Adaptive Difficulty: Uses machine-learning components to support adjustments in challenge difficulty based on user performance.
6. Habit Score and Analytics: Processes user activity and performance data to support habit scoring and progress tracking.
7. Recommendations: Supports personalized suggestions based on user behavior and performance.
8. Notifications: Integrates Firebase Cloud Messaging (FCM) to support alarm-related notifications.
9. API Communication: Provides API endpoints through which the React.js frontend communicates with backend services.

Database Integration

The backend integrates with PostgreSQL and MongoDB to manage application data.

- PostgreSQL: Used for structured information, including user accounts, alarm records, and related application data.

- MongoDB: Provides document-based storage for additional data where required by the application.

The backend communicates with these databases to retrieve, insert, update, and manage information as needed.

3.3 Front End

The frontend of the Intelligent Cognitive Alarm Platform is developed using React.js, JavaScript, HTML, and CSS. It provides an interactive user interface through which users can access the application's features and manage their alarms.

Main Functions of the Frontend

1. Login and Authentication: Provides the interface for users to log in and access their accounts.
2. Dashboard: Displays relevant alarm information, habit scores, performance details, and recommendations.
3. Alarm Management: Allows users to create, view, update, and delete alarms.
4. Cognitive Challenges: Displays questions and answer options when a cognitive challenge is presented.
5. Performance Tracking: Presents user activity, challenge results, and progress information.
6. Habit Score Display: Shows habit scores and related performance indicators.
7. Recommendations: Displays personalized suggestions to help users improve their wake-up habits.
8. Notifications: Supports browser notification functionality through Firebase Cloud Messaging, subject to permissions and configuration.

Frontend and Backend Communication

The frontend communicates with the backend through API requests. When a user performs an action, such as creating an alarm or submitting a challenge answer, the frontend sends the required information to the FastAPI backend. The backend processes the request and returns a response, which the frontend displays to the user.

Chapter 4

Snapshots

4.1 Front End

The front end of the Intelligent Cognitive Alarm Platform provides a user-friendly interface for managing alarms, monitoring sleep habits, viewing analytics, and tracking progress. It includes separate dashboards for users, administrators, and coaches, allowing each role to access relevant information and features.

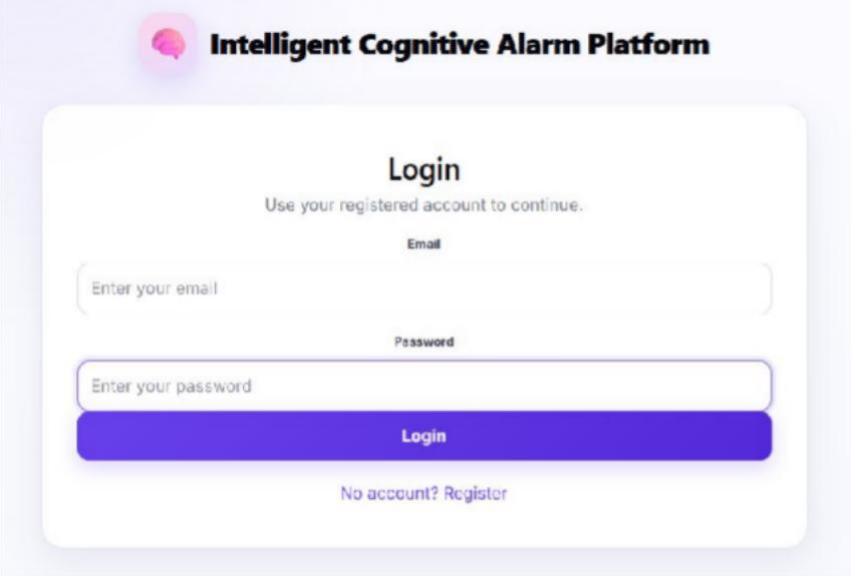
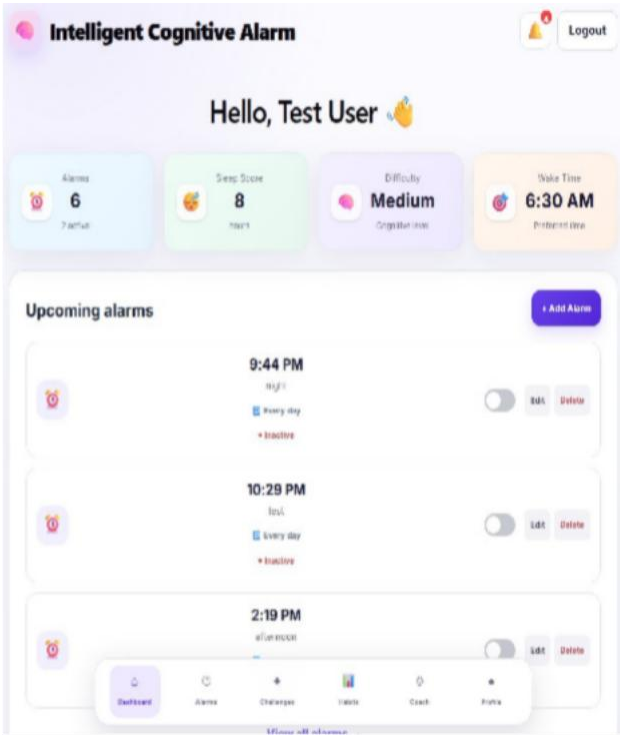


Fig 4.1.1 Login Page

Fig. 4.1.1 Login Page: The login page allows users to securely access the platform using their credentials. It provides an entry point to the system and helps ensure that users access the features associated with their accounts.

Fig. 4.1.2 User Dashboard: The user dashboard provides an overview of the user's alarm activities, wake-up performance, habit scores, and progress. It allows users to access important features and monitor their wake-up routines.



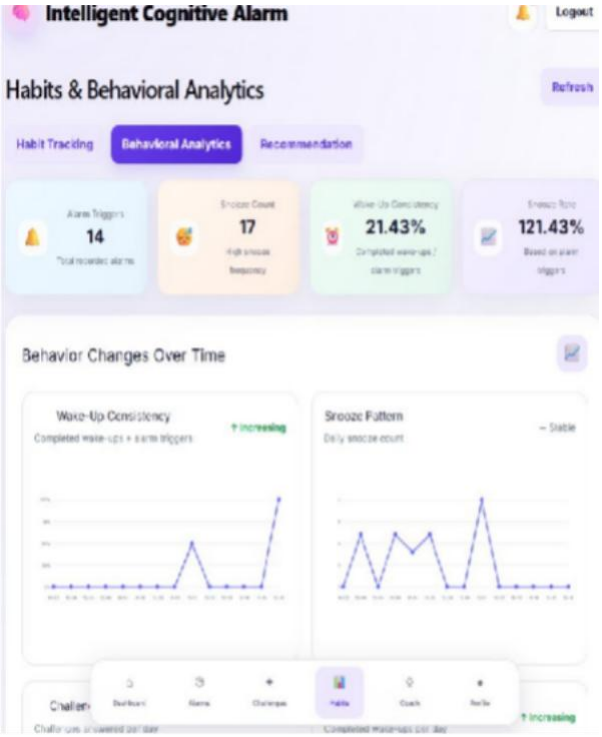


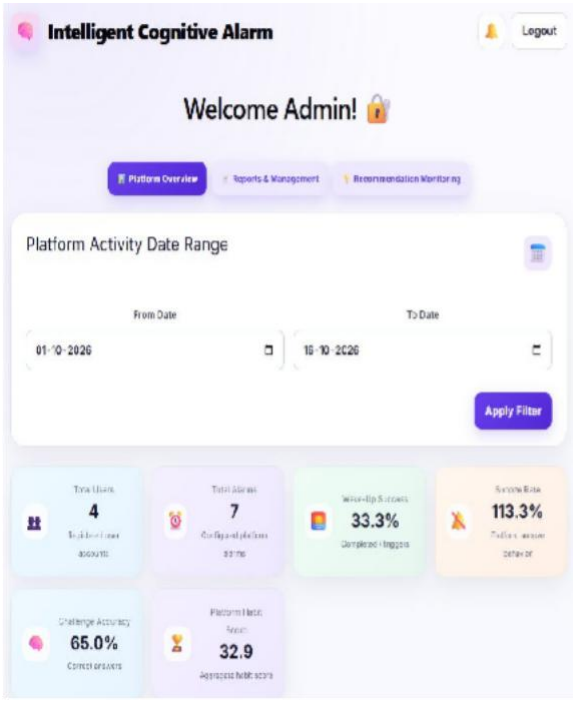
Fig 4.1.2 User dashboard **Fig 4.1.3 Analytics Page**

Fig. 4.1.3 Analytics Page: The analytics page presents information about the user's alarm and cognitive challenge performance. It helps users understand their progress through recorded performance data and visual summaries.

Fig. 4.1.4 Admin Dashboard: The admin dashboard provides an administrative view of the platform. It helps the administrator access relevant system information and manage platform-related activities according to the available functionality.

Fig. 4.1.5 Analytics: This section displays analytical information related to wake-up behaviour and challenge performance. It helps identify patterns in user activity and evaluate progress over time.

Fig. 4.1.6 Coach Dashboard: The coach dashboard provides a view of user progress and behavioural information to support monitoring and guidance. It helps coaches review relevant performance details and understand users' progress.



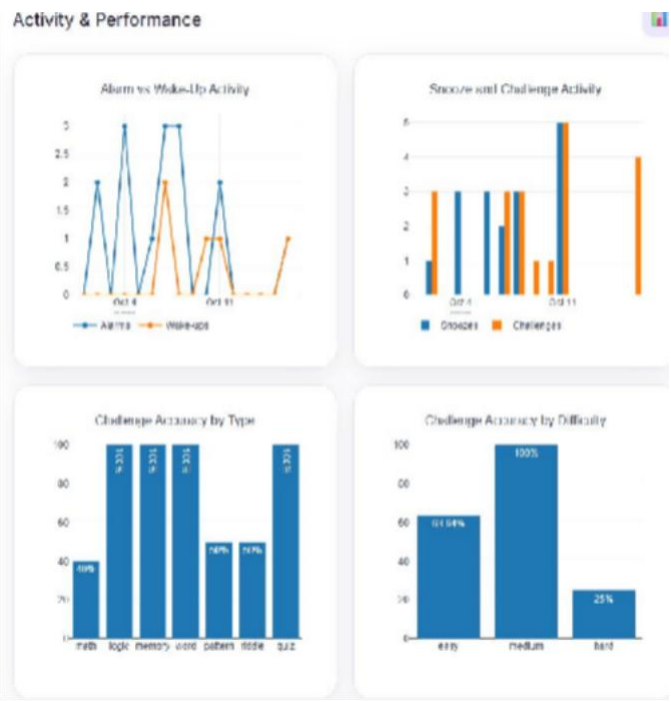


Fig 4.1.4 Admin Dashboard

Fig 4.1.5 Analytics

Fig. 4.1.7 Sleep & Progress Trend: This section presents sleep-related and progress trends over time. It helps users observe changes in their routines and evaluate improvements in wake-up consistency and overall habits.



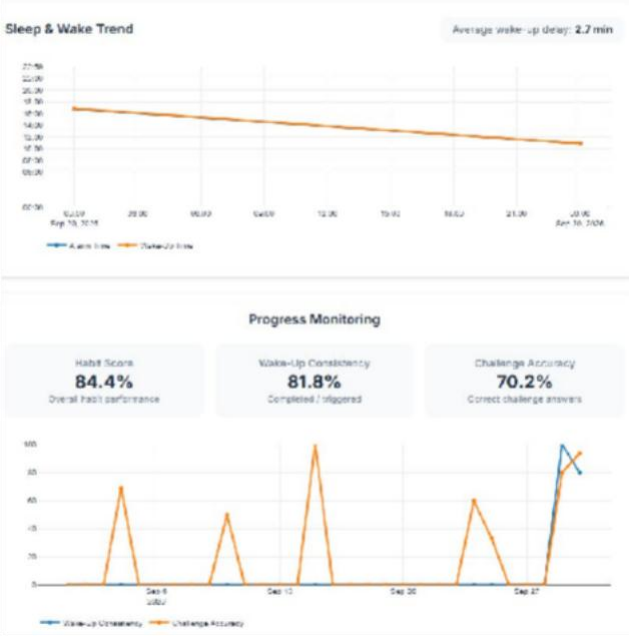


Fig 4.1.6 Coach Dashboard

Fig 4.1.7 Sleep & Progress Trend

4.2 Back End

The backend of the Intelligent Cognitive Alarm Platform is developed using Python and FastAPI. It processes requests received from the front end, manages application logic, communicates with the databases, and supports features such as alarm management, cognitive challenge processing, performance tracking, and analytics.



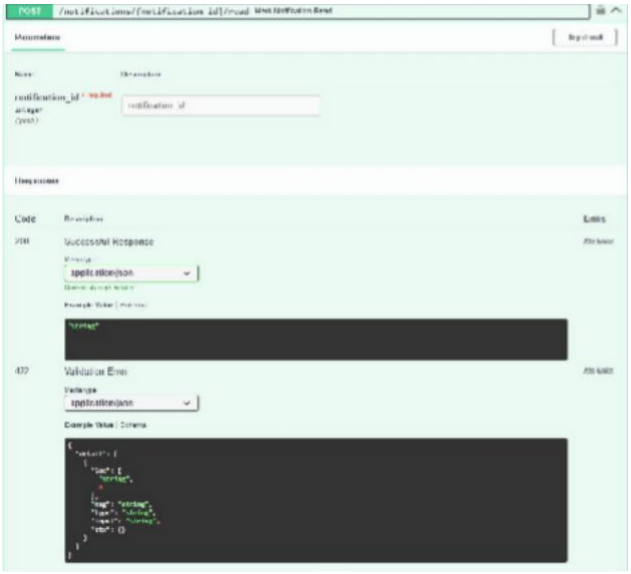


Fig 4.2.1 Fast API **Fig 4.2.2 Testing**

Fig. 4.2.1 FastAPI: The FastAPI interface provides access to the backend API endpoints. It allows developers to view the available API operations and inspect or test requests and responses. This interface helps verify communication between the front end and the backend services.

Testing: Backend testing is performed to check whether the API endpoints respond correctly to requests. Tools such as Postman can be used to send requests, examine responses, and verify API functionality. Testing helps identify errors and confirm that the backend operations work as expected.

4.3 Docker

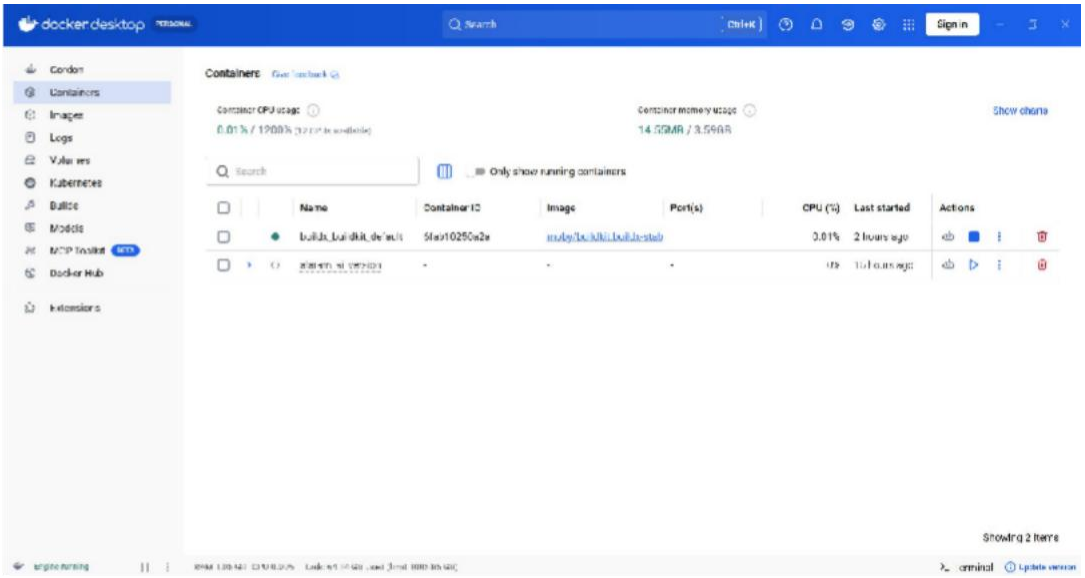


Fig. 4.3: Docker Desktop Showing the Application Container Status

Figure 4.3 shows the Docker Desktop interface used to monitor the containerized application. The Docker engine is running, and the alaram_ai_version project is listed in the Containers section. The interface allows developers to start, stop, and monitor application containers. The screenshot demonstrates the Docker environment used for deployment and management of the Intelligent Cognitive Alarm Platform.

CHAPTER 5

DEVELOPMENT

The development phase involves implementing the Intelligent Cognitive Alarm Platform using the selected frontend technologies, backend framework, databases, and AI/ML components. The system is developed as a combination of frontend interfaces and backend services that work together to provide alarm management, cognitive challenges, behavioural analysis, and progress tracking.

The frontend is developed using React.js, JavaScript, HTML, and CSS to provide an interactive user interface. The backend is implemented using Python and FastAPI to process user requests, manage alarms, evaluate cognitive challenge responses, and handle application logic.

PostgreSQL and MongoDB are used for data storage, while machine learning and data analysis components support adaptive difficulty, habit scoring, and personalized recommendations. Firebase Cloud Messaging (FCM) is integrated to support push notifications.

The code is organized into separate modules to improve readability, maintainability, and ease of testing. The application is tested using API testing tools and deployed locally using Docker and Docker Compose.

CHAPTER 6

DEPLOYMENT (FRONTEND AND BACKEND)

The Intelligent Cognitive Alarm Platform is deployed using Docker and Docker Compose. This approach allows the frontend, backend, and database services to run in separate containers and communicate with one another.

6.1 Frontend Deployment

The frontend is developed using React.js, JavaScript, HTML, and CSS. It provides the user interface for login, alarm management, dashboards, analytics, and progress monitoring.

The frontend is configured to run through Docker Compose, making it accessible through a web browser. Users can interact with the application and view information received from the backend.

6.2 Backend Deployment

The backend is developed using Python and FastAPI. It handles API requests, alarm-related operations, cognitive challenge processing, user data management, and analytical functions.

The backend runs in a Docker container and exposes its API through port 8000. The FastAPI documentation interface can be accessed at:

http://localhost:8000/docs

This interface allows developers to view the available API endpoints and test backend operations.

6.3 Database Deployment

The platform uses PostgreSQL and MongoDB for data storage. PostgreSQL runs on port 5432, while MongoDB runs on port 27017 in the local Docker setup.

Docker Compose helps manage the application services and their connections. The databases store the information required by the platform, depending on the implemented database operations.

6.4 Docker and Docker Compose

Docker is used to package the application components into containers. Docker Compose is used to build, start, and manage the frontend, backend, and database services together.

The following command is used to build and start the application:

docker compose up -d --build

The following command checks the status of the containers:

docker compose ps

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The Intelligent Cognitive Alarm Platform is designed to improve wake-up habits by combining alarm management with cognitive challenges, behavioural analysis, and personalized recommendations. Unlike traditional alarm systems, the platform encourages users to complete challenges before dismissing alarms, helping make the waking process more engaging.

The system provides features such as user authentication, alarm scheduling, cognitive challenge verification, performance tracking, habit scoring, analytics dashboards, and adaptive difficulty. The frontend is developed using React.js, while the backend uses Python and FastAPI. PostgreSQL and MongoDB support data storage, and AI/ML technologies are incorporated into the analytical and adaptive components.

Docker and Docker Compose are used to run and manage the application services in a local deployment environment. Overall, the platform provides a foundation for monitoring wake-up behaviour and supporting the development of more consistent sleep and waking routines.

7.2 Future Enhancements

The following enhancements can be considered for future development:

1. Mobile Application: Develop dedicated Android and iOS applications to make the platform more accessible on smartphones.
2. Advanced AI Personalization: Improve the adaptive system using additional behavioural data and model evaluation.
3. Wearable Device Integration: Integrate smartwatches and fitness bands to collect sleep duration and activity information.
4. Advanced Sleep Analysis: Introduce sleep-quality analysis using reliable sleep-related data sources.

5. Cloud Deployment: Deploy the application on a cloud platform to improve remote accessibility and availability.
6. Enhanced Notifications: Improve push notifications and add configurable reminders for alarms and sleep schedules.
7. Improved Analytics: Provide more detailed weekly and monthly reports, including long-term habit trends.
8. Security Enhancements: Strengthen authentication, data protection, access control, and secure configuration management. These enhancements could improve personalization, accessibility, reliability, and the overall usefulness of the platform.

REFERENCES

The following references are suitable for your Intelligent Cognitive Alarm Platform project report. They cover the technologies used for frontend development, backend services, databases, AI/ML, notifications, and deployment.

You can copy this section into your report and number the references as shown below.

1. React. React Documentation. Available at: https://react.dev/

2. FastAPI. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/

3. PostgreSQL Global Development Group. PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/

4. MongoDB, Inc. MongoDB Documentation. Available at: https://www.mongodb.com/docs/

5. Docker, Inc. Docker Documentation. Available at: https://docs.docker.com/

6. Docker, Inc. Docker Compose Documentation. Available at: https://docs.docker.com/compose/

7. Google Firebase. Firebase Cloud Messaging Documentation. Available at: https://firebase.google.com/docs/cloud-messaging

8. Scikit-learn Developers. Scikit-learn User Guide. Available at: https://scikit-learn.org/stable/user_guide.html

9. XGBoost Developers. XGBoost Documentation. Available at: https://xgboost.readthedocs.io/

10. TensorFlow. TensorFlow Documentation. Available at: https://www.tensorflow.org/guide

11. PyTorch. PyTorch Documentation. Available at: https://pytorch.org/docs/stable/

12. The pandas Development Team. pandas Documentation. Available at: https://pandas.pydata.org/docs/

13. NumPy Developers. NumPy Documentation. Available at: https://numpy.org/doc/

14. Chart.js Contributors. Chart.js Documentation. Available at: https://www.chartjs.org/docs/latest/

15. Plotly Technologies Inc. Plotly Python Documentation. Available at: https://plotly.com/python/

16. Postman, Inc. Postman Documentation. Available at: https://learning.postman.com/

17. GitHub, Inc. GitHub Documentation. Available at: https://docs.github.com/

18. Python Software Foundation. Python Documentation. Available at: https://docs.python.org/3/

19. Mozilla. MDN Web Docs: HTML, CSS, and JavaScript. Available at: https://developer.mozilla.org/

20. OpenJS Foundation. JSON Web Token (JWT) Introduction. Available at: https://jwt.io/introduction

These official documentation sources describe the technologies and tools relevant to the project.